

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:

Студент 4 курса

Группы ИИ-26

Рулько М.А.

Проверила:

Андренко К.В.

Цель работы:

Осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Ход работы

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Код программы:

```
"""Обучение кастомной СНС и DenseNet121 на CIFAR-100, сравнение с SOTA, инференс."""
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision.transforms as transforms
from matplotlib.patches import Patch
from PIL import Image
from torchvision.models import densenet121, DenseNet121_Weights

import config
from data import get_custom_loaders, get_pretrained_loaders

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=config.NUM_CLASSES):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, num_classes)
        self.dropout = nn.Dropout(0.4)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

def build_densenet121(num_classes=config.NUM_CLASSES):
    model = densenet121(weights=DenseNet121_Weights.IMAGENET1K_V1)
    for param in model.features.parameters():
```

```

        param.requires_grad = False
    model.classifier = nn.Linear(model.classifier.in_features, num_classes)
    return model

def count_params(model, trainable_only=False):
    return sum(p.numel() for p in model.parameters()
               if p.requires_grad or not trainable_only)

def run_epoch(model, loader, criterion, device, optimizer=None):
    train = optimizer is not None
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0

    with torch.enable_grad() if train else torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            if train:
                optimizer.zero_grad()
                outputs = model(images)
                loss = criterion(outputs, labels)
                if train:
                    loss.backward()
                    optimizer.step()
            total_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)

    return total_loss / total, correct / total

def fit(model, train_loader, test_loader, criterion, optimizer, device, n_epochs):
    history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

    for epoch in range(1, n_epochs + 1):
        train_loss, train_acc = run_epoch(model, train_loader, criterion, device, optimizer)
        test_loss, test_acc = run_epoch(model, test_loader, criterion, device)

        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["test_loss"].append(test_loss)
        history["test_acc"].append(test_acc)

        print(f"Эпоха {epoch:2d}/{n_epochs} | train loss {train_loss:.4f} acc {train_acc*100:5.2f}%
| "
            f"test loss {test_loss:.4f} acc {test_acc*100:5.2f}%")

    return history

def plot_history(history, title_prefix="DenseNet121"):
    epochs = range(1, len(history["train_loss"]) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    axes[0].plot(epochs, history["train_loss"], label="Train loss", marker='o', markersize=3)
    axes[0].plot(epochs, history["test_loss"], label="Test loss", marker='o', markersize=3)
    axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
    axes[0].set_title(f"{title_prefix}: изменение ошибки")
    axes[0].legend(); axes[0].grid(alpha=0.3)

    axes[1].plot(epochs, [a * 100 for a in history["train_acc"]], label="Train acc", marker='o',
markersize=3)
    axes[1].plot(epochs, [a * 100 for a in history["test_acc"]], label="Test acc", marker='o',
markersize=3)
    axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("Accuracy, %")
    axes[1].set_title(f"{title_prefix}: изменение точности")
    axes[1].legend(); axes[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.show()

FAIR_BASELINES = {
    "VGG-16": {"acc": 72.7, "params": 20_000_000, "note": "с нуля"},
    "ResNet-18": {"acc": 75.6, "params": 11_000_000, "note": "с нуля"},
    "ResNet-56": {"acc": 76.5, "params": 24_000_000, "note": "с нуля"},
    "Wide ResNet-28-10": {"acc": 81.2, "params": 36_000_000, "note": "с аугментацией"},
}

SOTA_PRETRAINED = {
    "DenseNet-BC (k=24)": {"acc": 82.8, "params": 27_000_000, "note": "ImageNet pretrain"},
    "EfficientNet-B7": {"acc": 87.3, "params": 66_000_000, "note": "ImageNet pretrain"},
    "FixResNeXt-101": {"acc": 89.9, "params": 84_000_000, "note": "ImageNet pretrain"},
    "ViT-L/16": {"acc": 93.0, "params": 307_000_000, "note": "JFT-300M pretrain"},
    "ViT-H/14": {"acc": 93.8, "params": 632_000_000, "note": "JFT-300M pretrain"},
}

```

```

def print_table(title, data):
    print(f"\n--- {title} ---")
    print(f"{'Модель':<28}{'Точность':>10}{'Параметры':>15}    Примечание")
    print("-" * 75)
    for name, info in data.items():
        print(f"{'name':<28}{'info['acc']':>9.2f}%{'info['params']':>15,}    {'info['note']}"")

def plot_comparison(your_results):
    all_models = (**your_results, **FAIR_BASELINES, **SOTA_PRETRAINED)
    names = list(all_models.keys())
    accs = [all_models[n]['acc'] for n in names]
    colors = (['crimson'] * len(your_results)
              + ['steelblue'] * len(FAIR_BASELINES)
              + ['seagreen'] * len(SOTA_PRETRAINED))

    fig, ax = plt.subplots(figsize=(12, 6))
    bars = ax.barh(names, accs, color=colors)
    ax.set_xlim(0, 102)
    ax.set_xlabel("Точность на CIFAR-100, %")
    ax.set_title("Сравнение ваших моделей с бейзлайнами и SOTA (CIFAR-100)")
    ax.invert_yaxis()
    ax.grid(axis='x', alpha=0.3)

    for bar, acc in zip(bars, accs):
        ax.text(acc + 0.3, bar.get_y() + bar.get_height() / 2,
                f"{'acc':.2f}%", va='center', fontsize=9)

    ax.legend(handles=[
        Patch(facecolor='crimson', label='Ваши модели'),
        Patch(facecolor='steelblue', label='Бейзлайны (с нуля)'),
        Patch(facecolor='seagreen', label='SOTA (с предобучением)'),
    ], loc='center right')
    plt.tight_layout()
    plt.show()

def compare_with_sota(history, history_pt, n_params, n_params_trainable, n_params_total):
    """Печатает таблицы, рисует диаграмму и выводит итоги."""
    print("Сравнение результатов:")
    print(f"Кастомная СНС: {history['test_acc'][-1]*100:.2f}% ({n_params:,} параметров, все обучаемые)")
    print(f"DenseNet121: {history_pt['test_acc'][-1]*100:.2f}% "
          f"({n_params_trainable:,} обучаемых параметров из {n_params_total:,})")

    print("\n" + "=" * 70)
    print("СПАВНЕНИЕ С SOTA НА CIFAR-100")
    print("=" * 70)

    your_results = {
        "SimpleCNN (ваша)": {"acc": history['test_acc'][-1] * 100, "params": n_params,
                             "note": f"с нуля, {len(history['test_acc'])} эпох"},
        "DenseNet121 (ваша)": {"acc": history_pt['test_acc'][-1] * 100, "params":
n_params_trainable,
                             "note": f"только классификатор, {len(history_pt['test_acc'])} эпохи"},
    }

    print_table("Ваши модели", your_results)
    print_table("Честные бейзлайны (обучение с нуля)", FAIR_BASELINES)
    print_table("SOTA с предобучением (нельзя сравнивать напрямую)", SOTA_PRETRAINED)

    plot_comparison(your_results)

    your_best = max(v['acc'] for v in your_results.values())
    fair_best = max(v['acc'] for v in FAIR_BASELINES.values())
    sota_best = max(v['acc'] for v in SOTA_PRETRAINED.values())

    print("\n" + "=" * 70)
    print("ИТОГИ")
    print("=" * 70)
    print(f"Лучшая ваша модель: {your_best:.2f}%")
    print(f"Лучший честный бейзлайн: {fair_best:.2f}% (отставание: {fair_best - your_best:.2f} п.п.)")
    print(f"Лучший SOTA (с предобучением): {sota_best:.2f}% (отставание: {sota_best - your_best:.2f} п.п.)")
    print("\nПримечание: SOTA-модели используют предобучение на")
    print("ImageNet/JFT-300M, поэтому их сравнение с моделями,")
    print("обученными с нуля, не является строгим.")

def predict_with_both(path, custom_model, pretrained_model, device):
    img = Image.open(path).convert('RGB')

    preprocess_custom = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),

```

```

        transforms.Normalize(config.MEAN, config.STD),
    ])
    preprocess_pt = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(config.IMAGENET_MEAN, config.IMAGENET_STD),
    ])

    custom_model.eval()
    pretrained_model.eval()

    with torch.no_grad():
        probs_custom = F.softmax(
            custom_model(preprocess_custom(img).unsqueeze(0).to(device)), dim=1
        ).cpu().numpy().flatten()
        probs_pt = F.softmax(
            pretrained_model(preprocess_pt(img).unsqueeze(0).to(device)), dim=1
        ).cpu().numpy().flatten()

    pred_custom = int(np.argmax(probs_custom))
    pred_pt = int(np.argmax(probs_pt))

    fig, axes = plt.subplots(1, 3, figsize=(15, 4), gridspec_kw={'width_ratios': [1, 1.2, 1.2]})
    axes[0].imshow(img); axes[0].axis('off'); axes[0].set_title("Входное изображение")

    axes[1].barh(config.CLASSES, probs_custom * 100,
                 color=['red' if i == pred_custom else 'steelblue' for i in
range(config.NUM_CLASSES)])
    axes[1].set_xlim(0, 100); axes[1].invert_yaxis()
    axes[1].set_title(f"Кастомная CHC: {config.CLASSES[pred_custom]}")
    axes[1].set_xlabel("Вероятность, %")

    axes[2].barh(config.CLASSES, probs_pt * 100,
                 color=['green' if i == pred_pt else 'steelblue' for i in
range(config.NUM_CLASSES)])
    axes[2].set_xlim(0, 100); axes[2].invert_yaxis()
    axes[2].set_title(f"DenseNet121: {config.CLASSES[pred_pt]}")
    axes[2].set_xlabel("Вероятность, %")

    plt.tight_layout()
    plt.show()
    return pred_custom, pred_pt

def main():
    config.set_seed()
    device = config.get_device()
    print("Используемое устройство:", device)

    train_loader, test_loader = get_custom_loaders()

    custom_model = SimpleCNN().to(device)
    n_params = count_params(custom_model, trainable_only=True)
    print(custom_model)
    print(f"Обучаемых параметров: {n_params:,}")

    history = fit(
        custom_model, train_loader, test_loader,
        criterion=nn.CrossEntropyLoss(),
        optimizer=optim.Adam(custom_model.parameters(), lr=config.LR),
        device=device, n_epochs=config.N_EPOCHS,
    )

    train_loader_pt, test_loader_pt = get_pretrained_loaders()

    pretrained_model = build_densenet121().to(device)
    n_params_total = count_params(pretrained_model)
    n_params_trainable = count_params(pretrained_model, trainable_only=True)
    print(pretrained_model.classifier)
    print(f"Всего параметров: {n_params_total:,}")
    print(f"Обучаемых параметров (только классификатор): {n_params_trainable:,}")

    history_pt = fit(
        pretrained_model, train_loader_pt, test_loader_pt,
        criterion=nn.CrossEntropyLoss(),
        optimizer=optim.Adam(filter(lambda p: p.requires_grad, pretrained_model.parameters()),
                                lr=config.LR_PT),
        device=device, n_epochs=config.N_EPOCHS_PT,
    )

    plot_history(history_pt, title_prefix="DenseNet121")
    print(f"Итоговая точность DenseNet121 на тесте: {history_pt['test_acc'][-1]*100:.2f}%")

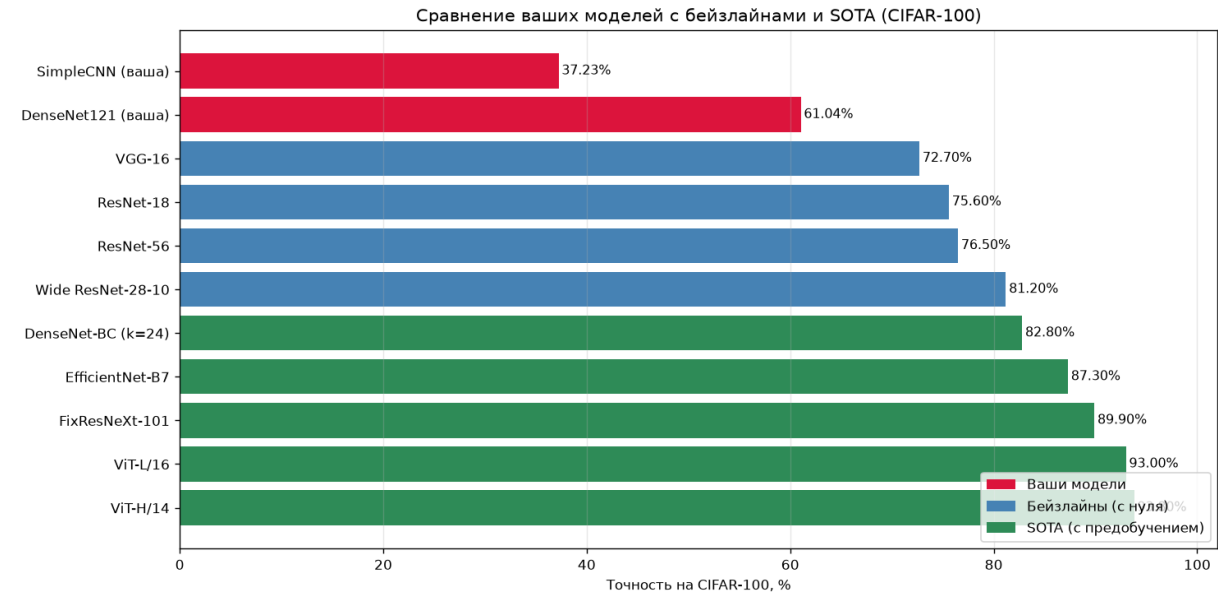
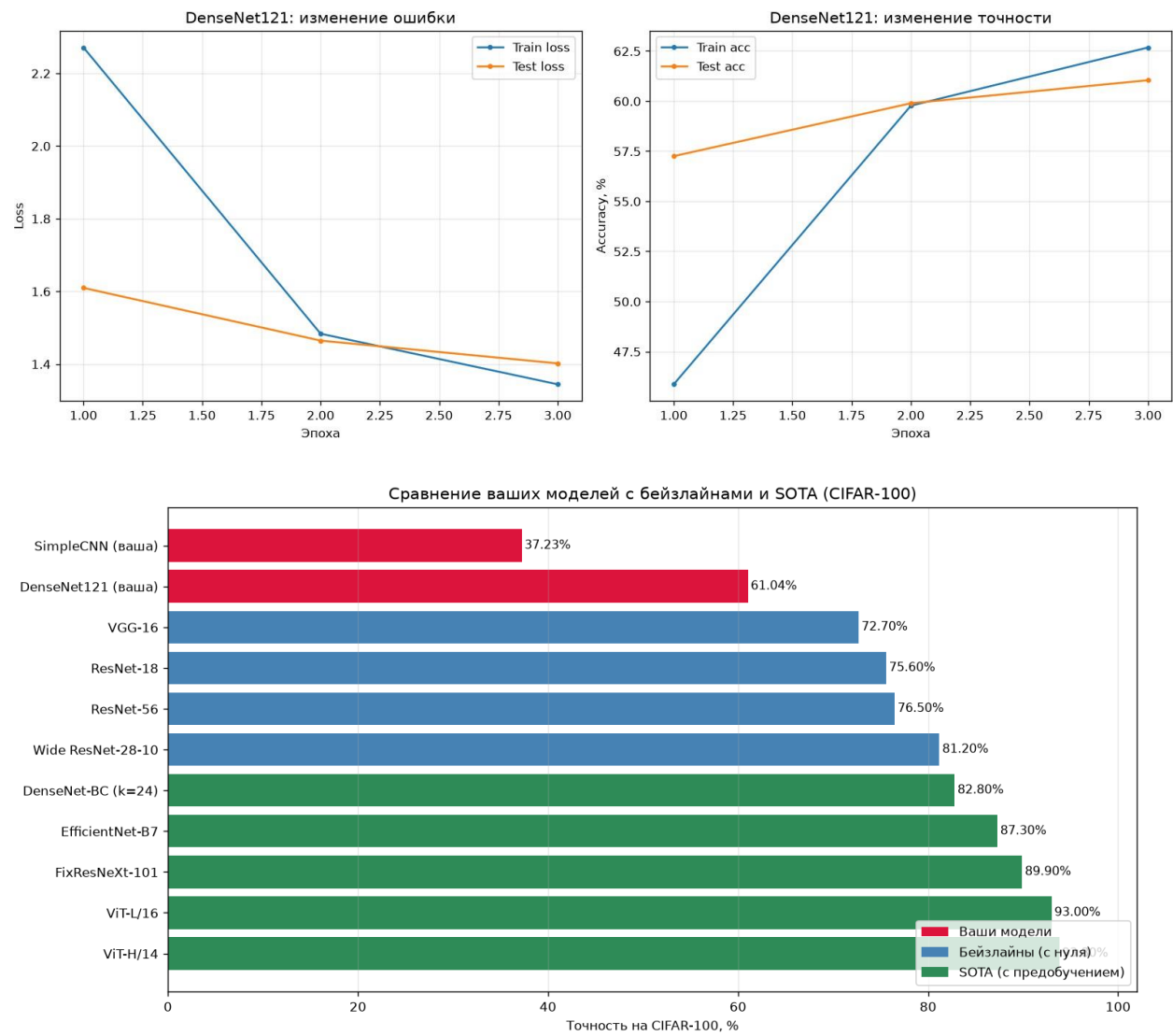
    compare_with_sota(history, history_pt, n_params, n_params_trainable, n_params_total)

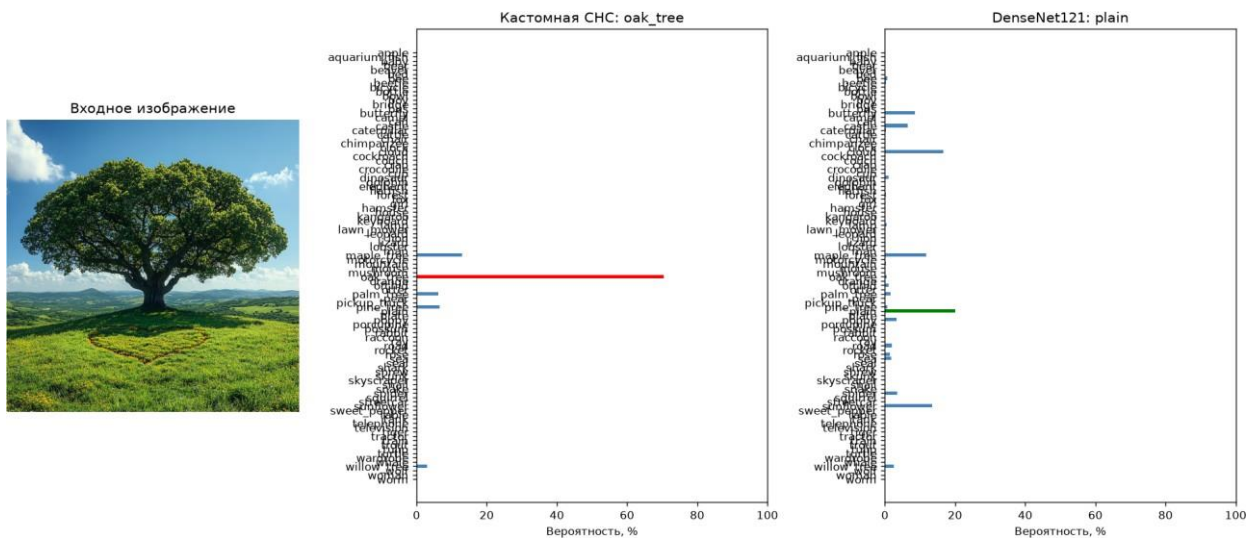
```

```
    predict_with_both(config.TEST_IMAGE_PATH, custom_model, pretrained_model, device)

if __name__ == "__main__":
    main()
```

Результат программы:





Устройство: cpu

1. SimpleCNN с нуля (643,492 параметров)

Данные: train 50000 | test 10000 | вход 3x32x32

[1/8] train 4.0635 / 7.29% | test 3.5557 / 15.78% | 42 с

...

2. DenseNet121: обучается только классификатор (102,500 из 7,056,356 параметров)

Данные: train 50000 | test 10000 | вход 3x224x224

[1/3] train 2.2708 / 45.86% | test 1.6093 / 57.25% | 310 с

...

СРАВНЕНИЕ С БЕЙЗЛАЙНАМИ И SOTA (CIFAR-100)

Модель	Точность	Параметры	Примечание
--------	----------	-----------	------------

ViT-H/14	93.80%	632,000,000	JFT-300M pretrain
----------	--------	-------------	-------------------

...

* DenseNet121 (ваша)	61.04%	102,500	только классификатор, 3 эпохи
----------------------	--------	---------	-------------------------------

* SimpleCNN (ваша)	37.23%	643,492	с нуля, 8 эпох
--------------------	--------	---------	----------------

* - ваши модели. Модели с предобучением нельзя строго сравнивать с обучением с нуля.

Лучшая ваша модель: 61.04% | отставание от бейзлайна с нуля: 20.16 п.п. | от SOTA: 32.76 п.п.

ИТОГИ

Лучшая ваша модель:	61.04%
Лучший честный бейзлайн: 81.20% (отставание: +20.16 п.п.) Лучший SOTA (с предобучением): 93.80% (отставание: +32.76 п.п.) Примечание: SOTA-модели используют предобучение на ImageNet/JFT-300M, поэтому их сравнение с моделями, обученными с нуля, не является строгим.	

Вывод:

В ходе лабораторной работы обучена предобученная DenseNet121 на CIFAR-100 с оптимизатором Adam. Классификатор заменён на Linear(1024, 100) под число классов, изображения масштабированы с 32×32 до 224×224 с нормализацией ImageNet.

Backbone заморожен, обучался только классификатор (102 500 параметров из 7 056 356). За 3 эпохи с CrossEntropyLoss достигнута точность 61.04% на тесте; графики показали устойчивое снижение ошибки и рост точности без переобучения. Кастомная SimpleCNN из ЛР №1 (643 492 параметра, 8 эпох с нуля) дала 37.23%: DenseNet121 превзошла её на 23.8 п.п. при примерно в 6 раз меньшем числе обучаемых параметров, что подтверждает эффективность transfer learning. Сравнение с литературными результатами показало отставание на 12-20 п.п. от честных бейзлайнов, обученных с нуля (Wide ResNet-28-10 — 81.2%), и на 22-33 п.п. от моделей с предобучением (ViT-H/14 — 93.8%). Это объясняется замороженным backbone, малым числом эпох и отсутствием сильных аугментаций. Реализована визуализация работы обеих сетей на произвольном изображении с отображением распределения вероятностей по классам.